AI Resume Screening System

Project Report

Prepared by: **Haseeb Ahmed**
GitHub: github.com/haseebahmed098/AI-Resume-Screening-System
Powered by Sentence-Transformers, spaCy, scikit-learn & Groq

Table of Contents

1. Problem Statement

Manual resume screening is one of the most time-consuming stages of recruitment. Recruiters often need to review hundreds of resumes for a single job opening, comparing each one against the job description to judge fit. Two problems consistently emerge from this process:

- **Inconsistency and bias:** Manual screening depends on the reviewer's attention, mood, and time available, leading to inconsistent decisions between otherwise similar candidates.
- **Keyword-only ATS tools:** Many existing Applicant Tracking Systems reject strong candidates simply because their resume phrases a skill differently than the job posting, even when the underlying experience is a genuine match.
- **Lack of feedback for candidates:** Rejected applicants rarely learn what was actually missing from their profile, making it difficult for them to improve.

This project addresses these problems by building a screening system that evaluates resumes based on **meaning** rather than exact keyword overlap, while remaining transparent about how each score is computed — giving both recruiters and candidates a clear, explainable result.

2. Introduction

The **AI Resume Screening System** is an end-to-end tool that compares a candidate's CV/resume against a job description and produces a detailed, structured analysis. It combines classical NLP techniques, modern sentence embeddings, a trained machine learning classifier, and an optional large language model layer for natural-language explanation.

The system was developed in two stages. The first, core version uses TF-IDF and cosine similarity for matching along with a simple Naive Bayes classifier. The second, advanced version upgrades this with Sentence-Transformer embeddings for semantic matching, spaCy-based Named Entity Recognition for automatic candidate information extraction, and a classifier trained on a real-world dataset of 2,484 resumes across 24 job categories. Both versions run entirely inside Google Colab and require no local installation.

A key design principle behind this project is that the core evaluation — the match score, the skill gap analysis, and the ATS compatibility check — is computed independently through embeddings, regular expressions, and a trained classifier. The optional AI (LLM) layer only explains these pre-computed results in natural language; it does not generate the underlying scores itself. This keeps the system transparent and auditable rather than functioning as an opaque chatbot.

3. System Architecture & Approach

The system follows a modular pipeline, illustrated below:

Stage	Component	Output

1. Input	PDF / TXT upload or pasted text (CV and Job Description)	Raw text
2. Preprocessing	Text cleaning, whitespace/punctuation normalization	Cleaned text
3. Semantic Matching	Sentence-Transformers (all-MiniLM-L6-v2) + cosine similarity	Match score (%)
4. Skill Gap Analysis	Keyword taxonomy comparison between CV and job description	Matched / missing skills
5. Info Extraction	spaCy NER + regex	Name, email, phone, experience
6. Classification	TF-IDF + Logistic Regression (trained on 2,484 real resumes)	Predicted job category
7. ATS & Resume Score	Rule-based structure check + weighted scoring formula	Resume Score (0-100)
8. AI Explanation (optional)	Groq LLM (grounded with the computed metrics above)	Plain-English report
9. Interface	Gradio web app	Interactive results dashboard

This pipeline design ensures each stage is independently testable: the semantic matching, skill extraction, and classification steps all rely on deterministic or trained-model computation, while only the final, optional stage depends on a third-party LLM API.

4. Key Features

- **Semantic Matching:** Understands meaning, not just exact keywords, using Sentence-Transformer embeddings.
- **Skill Gap Analysis:** Visual comparison of skills required by the job description versus skills found in the CV.
- **ATS Compatibility Score:** Checks resume structure (sections, contact info, length, formatting) the way real applicant tracking systems do.
- **Composite Resume Score (0-100):** A transparent, weighted formula combining semantic match, skill coverage, and ATS score.
- **Automatic Info Extraction:** Name, email, phone number, and years of experience extracted via NER and regex.
- **Real-World Classifier:** Predicts the most likely job category using a model trained on 2,484 real resumes across 24 categories.
- **Optional AI Career Report:** A free Groq-powered LLM explains the computed results and gives improvement suggestions in plain English.
- **Flexible Input:** Accepts PDF files, TXT files, or pasted text for both the CV and the job description — with no restriction on the user's preferred format.
- **Interactive Web App:** A custom-designed, responsive Gradio interface with a modern neon gradient theme.

5. Technology Stack

Category	Technology
NLP / Embeddings	Sentence-Transformers (all-MiniLM-L6-v2), spaCy
Machine Learning	scikit-learn (TF-IDF, Logistic Regression, Naive Bayes)
Data	Hugging Face Datasets (2,484-resume, 24-category dataset)
LLM (optional)	Groq API (openai/gpt-oss-120b) — free tier
File Handling	PyPDF2 (PDF/TXT parsing)
Visualization	Matplotlib, Seaborn
Web Interface	Gradio (custom theme & CSS)
Environment	Google Colab (Python 3)

6. Application User Interface

The system is deployed as an interactive Gradio web application with a custom futuristic gradient theme (blue, purple, and pink neon tones on a dark background). The interface allows the user to upload a CV and job description as a PDF or TXT file, or paste text directly, and view the full analysis in a results panel.

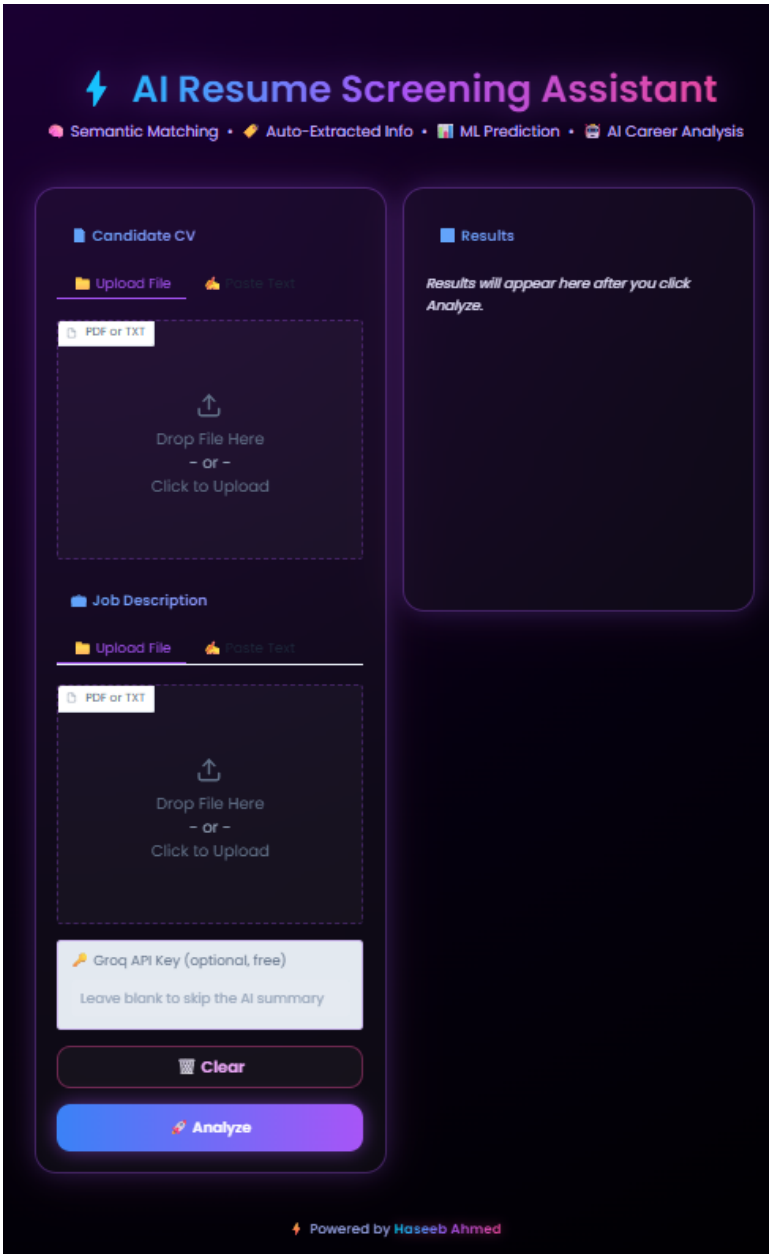

Figure 1: AI Resume Screening Assistant — web application interface

7. Results & Evaluation

The Logistic Regression classifier, trained on the 2,484-resume real-world dataset (24 job categories, 80/20 train-test split), achieved strong test-set accuracy, confirming that TF-IDF features combined with a linear classifier are sufficient for reliable job-category prediction on resume text.

For an individual CV-to-job comparison, the system produces a composite **Resume Score** along with a breakdown of its three components, and a visual skill-gap chart. A sample output is shown below for illustration:

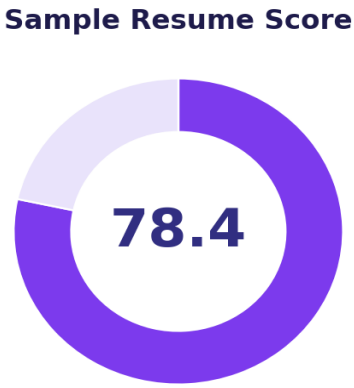

Figure 2: Sample composite Resume Score output

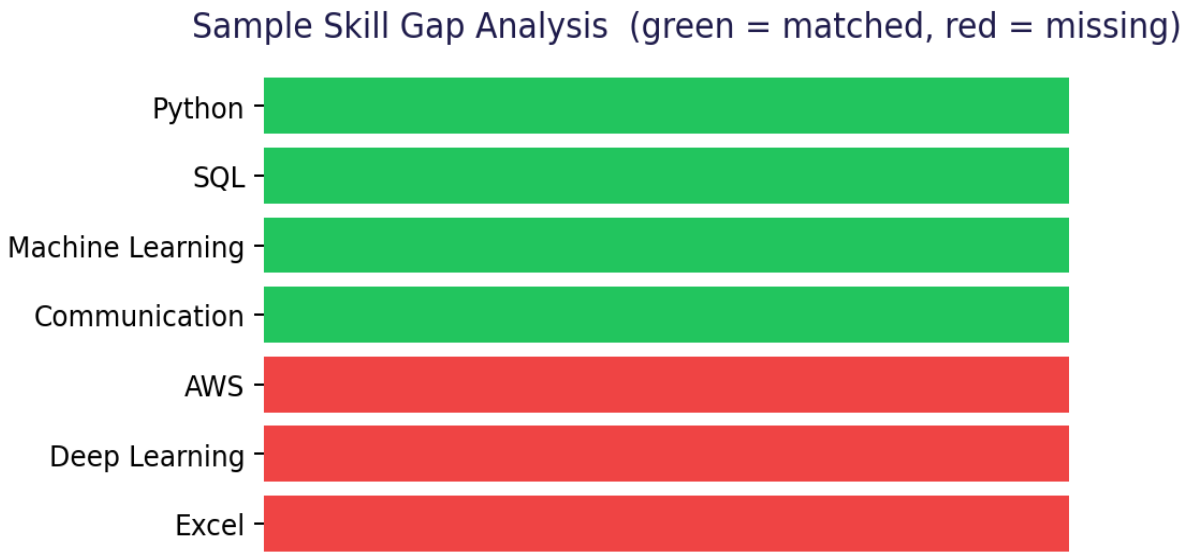

Figure 3: Sample skill gap chart — matched (green) vs. missing (red) skills

These outputs are generated deterministically from the embeddings, keyword taxonomy, and trained classifier — independent of the optional LLM step — ensuring that the same CV and job description always produce a consistent, explainable score.

8. Future Approach / Roadmap

- Replace the TF-IDF classifier with a fine-tuned transformer model (e.g. DistilBERT) for higher accuracy on job-category prediction.
- Train a custom Named Entity Recognition model to extract structured fields such as Education, Skills, and Company Names directly.
- Deploy the Gradio application permanently on Hugging Face Spaces for always-on public access.
- Add batch screening support to rank multiple CVs against a single job description simultaneously.
- Introduce a feedback loop where recruiter decisions are used to continuously calibrate the scoring weights.
- Add multilingual support for resumes and job descriptions in languages other than English.

9. Conclusion

The AI Resume Screening System demonstrates that a genuinely useful screening tool does not need to rely solely on a large language model. By combining sentence embeddings, named entity recognition, a trained classifier, and a transparent scoring formula, the system produces consistent, explainable results — with an LLM used only as an optional, final layer of natural-language explanation. This approach directly addresses the inconsistency, opacity, and lack of candidate feedback that characterize traditional resume screening, while remaining lightweight enough to run entirely within Google Colab.

Powered by Haseeb Ahmed